# P2079R4
# System execution context

Published Proposal, 2024-05-21

**This version:**
http://wg21.link/P2079R4

**Authors:**
Lee Howes
Ruslan Arutyunyan (Intel)
Michael Voss (Intel)
Lucian Radu Teodorescu

**Audience:**
SG1, LEWG

**Project:**
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

---

## Abstract

A standard execution context based on the facilities in [P2300R9] that implements parallel-forward-progress to maximise portability. A set of `system_context`s share an underlying shared thread pool implementation, and may provide an interface to an OS-provided system thread pool.

## Table of Contents

# § 1. Changes

## § 1.1. R4

- Add more design considerations & goals.
- Add comparison of different replaceability options
- Add motivation for replaceabiilty ABI standardization
- Add the example of the ABI for replacement
- Strengthen the lifetime guarantees.

## § 1.2. R3

- Remove `execute_all` and `execute_chunk`. Replace with compile-time customization and a design discussion.
- Add design discussion about the approach we should take for customization and the extent to which the context should be implementation-defined.
- Add design discussion for an explicit `system_context` class.
- Add design discussion about priorities.

## § 1.3. R2

- Significant redesign to fit in [P2300R9] model.
- Strictly limit to parallel progress without control over the level of parallelism.
- Remove direct support for task groups, delegating that to `async_scope`.

### 1.4. R1

- Minor modifications

### 1.5. R0

- First revision

## 2. Introduction

[P2300R9] describes a rounded set of primitives for asynchronous and parallel execution that give a firm grounding for the future. However, the paper lacks a standard execution context and scheduler. It has been broadly accepted that we need some sort of standard scheduler.

As part of [P3109R0], `system_context` was voted as a must-have for the initial release of senders/receivers. It provides a convenient and scalable way of spawning concurrent work for the users of senders/receivers.

As noted in [P2079R1], an earlier revision of this paper, the `static_thread_pool` included in later revisions of [P0443R14] had many shortcomings. This was removed from [P2300R9] based on that and other input.

One of the biggest problems with local thread pools is that they lead to CPU oversubscription. This introduces a performance problem for complex systems that are composed from many independent parts.

This revision updates [P2079R1] to match the structure of [P2300R9]. It aims to provide a simple, flexible, standard execution context that should be used as the basis for examples but should also scale for practical use cases. It is a minimal design, with few constraints, and as such should be efficient to implement on top of something like a static thread pool, but also on top of system thread pools where fixing the number of threads diverges from efficient implementation goals.

Unlike in earlier versions of this paper, we do not provide support for waiting on groups of tasks, delegating that to the separate `async_scope` design in [P3149R2], because that is not functionality specific to a system context. Lifetime management in general should be considered delegated to `async_scope`.

The system context is of undefined size, supporting explicitly *parallel forward progress*. By requiring only parallel forward progress, any created parallel context is able to be a view onto the underlying

shared global context. All instances of the `system_context` share the same underlying execution context. If the underlying context is a static thread pool, then all `system_context`s should reference that same static thread pool. This is important to ensure that applications can rely on constructing `system_context`s as necessary, without spawning an ever increasing number of threads. It also means that there is no isolation between `system_context` instances, which people should be aware of when they use this functionality. Note that if they rely strictly on parallel forward progress, this is not a problem, and is generally a safe way to develop applications.

The minimal extensions to basic parallel forward progress are to support fundamental functionality that is necessary to make parallel algorithms work:

- Cancellation: work submitted through the parallel context must be cancellable.
- Forward progress delegation: we must be able to implement a blocking operation that ensures forward progress of a complex parallel algorithm without special cases.

An implementation of `system_context` *should* allow link-time or run-time replacement of the implementation such that the context may be replaced with an implementation that compiles and runs in a single-threaded process or that can be replaced with an appropriately configured system thread pool by an end-user.

Some key concerns of this design are:

- Extensibility: being able to extend the design to work with new additions to the senders/receivers framework.
- Replaceability: allowing users to replace the default behavior provided by `system_context`.
- Shareability: being able to share this system context across all binaries in the same process.
- Lifetime: as `system_context` is a global resource, we need to pay attention to the lifetime of this resource.
- Performance: as we envision this to be used in many cases to spawn concurrent work, performance considerations are important.

# § 3. Design

## § 3.1. `system_context`

The `system_context` creates a view on some underlying execution context supporting *parallel forward progress*, with at least one thread of execution (which may be the main thread). A `system_context` must outlive any work launched on it.

```
class system_context {
public:
  system_context();
   ~system_context();

  system_context(const system_context&) = delete;
  system_context(system_context&&) = delete;
  system_context& operator=(const system_context&) = delete;
  system_context& operator=(system_context&&) = delete;

  implementation-defined-system_scheduler get_scheduler();
  size_t max_concurrency() const noexcept;
};
```

- On construction, the `system_context` may initialize a shared system context, if it has not been previously initialized.
- To support sharing of an underlying system context, two `system_context` objects do not guarantee task isolation. If work submitted by one can consume the thread pool, that can block progress of another.
- The `system_context` is non-copyable and non-moveable.
- The `system_context` must outlive work launched on it. If there is outstanding work at the point of destruction, `std::terminate` will be called.
- The `system_context` must outlive schedulers obtained from it. If there are outstanding schedulers at destruction time, this is undefined behavior.
- The lifetime of a `system_context` must be fully contained in the lifetime of `main()`.
- `get_scheduler` returns a `system_scheduler` instance that holds a reference to the `system_context`.
- `max_concurrency` will return a value representing the maximum number of threads the context may support. This is not a snapshot of the current number of threads, and may return

`numeric_limits<size_t>::max`.

## § 3.2. `system_scheduler`

A `system_scheduler` is a copyable handle to a `system_context`. It is the means through which agents are launched on a `system_context`. The `system_scheduler` instance does not have to outlive work submitted to it. The `system_scheduler` is technically implementation-defined, but must be nameable. See later discussion for how this might work.

```
class implementation-defined-system_scheduler {
public:
  system_scheduler() = delete;
  ~system_scheduler();

  system_scheduler(const system_scheduler&);
  system_scheduler(system_scheduler&&);
  system_scheduler& operator=(const system_scheduler&);
  system_scheduler& operator=(system_scheduler&&);

  bool operator==(const system_scheduler&) const noexcept;

  friend implementation-defined-system_sender tag_invoke(
    std::execution::schedule_t, const implementation-defined-system_schedu
  friend std::execution::forward_progress_guarantee tag_invoke(
    std::execution::get_forward_progress_guarantee_t,
    const system_scheduler&) noexcept;
  friend implementation-defined-bulk-sender tag_invoke(
    std::execution::bulk_t,
    const system_scheduler&,
    Sh&& sh,
    F&& f) noexcept;
};
```

- `system_scheduler` is not independently constructible, and must be obtained from a `system_context`. It is both move and copy constructible and assignable.
- Two `system_scheduler`s compare equal if they share the same underlying implementation of `system_context` (e.g., they can compare equal, even if they were generated from two different `system_context` objects).

- A `system_scheduler` has reference semantics with respect to its `system_context`. Calling any operation other than the destructor on a `system_scheduler` after the `system_context` it was created from is destroyed is undefined behavior, and that operation may access freed memory.
- The `system_scheduler`:
  - satisfies the `scheduler` concept and implements the `schedule` customisation point to return an `implementation-defined` `sender` type.
  - implements the `get_forward_progress_guarantee` query to return `parallel`.
  - implements the `bulk` CPO to customise the `bulk` sender adapter such that:
    - When `execution::set_value(r, args...)` is called on the created `receiver`, an agent is created with parallel forward progress on the underlying `system_context` for each `i` of type `Shape` from `0` to `sh`, where `sh` is the shape parameter to the bulk call, that calls `f(i, args...)`.
- `schedule` calls on a `system_scheduler` are non-blocking operations.
- If the underlying `system_context` is unable to make progress on work created through `system_scheduler` instances, and the sender retrieved from `scheduler` is connected to a `receiver` that supports the `get_delegatee_scheduler` query, work may scheduled on the `scheduler` returned by `get_delegatee_scheduler` at the time of the call to `start`, or at any later point before the work completes.

## § 3.3. `system_sender`

```
class implementation-defined-system_sender {
public:
  friend pair<std::execution::system_scheduler, delegatee_scheduler> tag_i
    std::execution::get_completion_scheduler_t<set_value_t>,
    const system_scheduler&) noexcept;
  friend pair<std::execution::system_scheduler, delegatee_scheduler> tag_i
    std::execution::get_completion_scheduler_t<set_stopped_t>,
    const system_scheduler&) noexcept;

  template<receiver R>
        requires receiver_of<R>
  friend implementation-defined-operation_state
    tag_invoke(execution::connect_t, implementation-defined-system_sender&
```

```
  ...
};
```

`schedule` on a `system_scheduler` returns some implementation-defined `sender` type.

This sender satisfies the following properties:

- Implements the `get_completion_scheduler` query for the value and done channel where it returns a type that is logically a pair of an object that compares equal to itself, and a representation of delegatee scheduler that may be obtained from receivers connected with the sender.
- If connected with a `receiver` that supports the `get_stop_token` query, if that `stop_token` is stopped, operations on which `start` has been called, but are not yet running (and are hence not yet guaranteed to make progress) **must** complete with `set_stopped` as soon as is practical.
- `connect`ing the `sender` and calling `start()` on the resulting operation state are non-blocking operations.

# § 4. Design discussion and decisions

## § 4.1. To drive or not to drive

On single-threaded systems (e.g., freestanding implementations) or on systems in which the main thread has special significance (e.g., to run the Qt main loop), it's important to allow scheduling work on the main thread. For this, we need the main thread to *drive* work execution.

The earlier version of this paper, [P2079R2], included `execute_all` and `execute_chunk` operations to integrate with senders. In this version we have removed them because they imply certain requirements of forward progress delegation on the system context and it is not clear whether or not they should be called. We envision a separate paper that adds the support for drive-ability, which is decoupled by this paper.

We can simplify this discussion to a single function:

```
void drive(system_context& ctx, sender auto snd);
```

Let's assume we have a single-threaded environment, and a means of customising the `system_context` for this environment. We know we need a way to donate `main`'s thread to this

context, it is the only thread we have available. Assuming that we want a `drive` operation in some form, our choices are to:

- define our `drive` operation, so that it is standard, and we use it on this system.
- or allow the customisation to define a custom `drive` operation related to the specific single-threaded environment.

With a standard `drive` of this sort (or of the more complex design in [P2079R2]) we might write an example to use it directly:

```
system_context ctx;
auto snd = on(ctx, doWork());
drive(ctx, std::move(snd));
```

Without drive, we rely on an `async_scope` to spawn the work and some system-specific drive operation:

```
system_context ctx;
async_scope scope;
auto snd = on(ctx, doWork());
scope.spawn(std::move(snd));
custom_drive_operation(ctx);
```

Neither of the two variants is very portable. The first variant requires applications that don't care about drive-ability to call `drive`, while the second variant requires custom pluming to tie the main thread with the system scheduler.

We envision a new paper that adds support for a *main scheduler* similar to the *system scheduler*. The main scheduler, for hosted implementations would be typically different than the system scheduler. On the other hand, on freestanding implementations, the main scheduler and system scheduler can share the same underlying implementation, and both of them can execute work on the main thread; in this mode, the main scheduler is required to be driven, so that system scheduler can execute work.

Keeping those two topic as separate papers allows to make progress independently.

## § 4.2. Freestanding implementations

This paper payed attention to freestanding implementations, but doesn't make any wording proposals for them. We express a strong desire for the system scheduler to work on freestanding implementa-

tions, but leave the details to a different paper.

We envision that, a followup specification will ensure that the system scheduler will work in free-standing implementations by sharing the implementation with the main scheduler, which is driven by the main thread.

## § 4.3. Making `system_context` implementation-defined and replaceable

The system context aims to allow people to implement an application that is dependent only on parallel forward progress and to port it to a wide range of systems. As long as an application does not rely on concurrency, and restricts itself to only the system context, we should be able to scale from single threaded systems to highly parallel systems.

In the extreme, this might mean porting to an embedded system with a very specific idea of an execution context. Such a system might not have a multi-threading support at all, and thus the system context not only runs with single thread, but actually runs on the system's only thread. We might build the context on top of a UI thread, or we might want to swap out the system-provided implementation with one from a vendor (like Intel) with experience writing optimised threading runtimes.

The latter is also important for the composability of the existing code with the `system_context`, i.e., if Intel Threading building blocks (oneTBB) is used by somebody and they want to start using `system_context` as well, it's likely that the users want to replace `system_context` implementation with oneTBB because in that case they would have one thread pool and work scheduler underneath.

We need to allow customisation of the system context to cover this full range of cases. For a whole platform this is relatively simple. We assume that everything is an implementation-defined type. The `system_context` itself is a named type, but in practice is implementation-defined, in the same way that `std::vector` is implementation-defined at the platform level.

Other situations may offer a little less control. If we wish Intel to be able to replace the system thread pool with TBB, or Adobe to customise the runtime that they use for all of Photoshop to adapt to their needs, we need a different customisation mechanism.

To achieve this we see options:

1. Link-time replaceability. This could be achieved using weak symbols, or by choosing a runtime library to pull in using build options.
2. Run-time replaceability. This could be achieved by subclassing and requiring certain calls to be made early in the process.

3. Compile-time replaceability. This could be achieved by importing different headers, by macro definitions on the command line or various other mechanisms.

Link-time replaceability has the following characteristics:

- Pro: we have precedence in the standard: this is similar to replacing `operator new`.
- Pro: more predictable, in that it can be guaranteed to be application-global.
- Pro: some of the type erasure and indirection can be removed in practice with link-time optimisation.
- Con: it requires defining the ABI and thus, in some cases, would require some type erasure and some inefficiency.
- Con: harder to get it correctly with shared libraries (e.g., DLLs might have different replaced versions of the system scheduler).
- Con: the replacement might depend on the order of linking.

Run-time replaceability has the following characteristics:

- Pro: we have precedence in the standard: this is similar to `std::set_terminate()`.
- Pro: easier to achieve consistent behavior on applications with shared libraries (e.g., Windows has the same version of C++ standard library in DLL).
- Pro: a program can have multiple implementations of system scheduler.
- Con: race conditions between replacing the system scheduler and using it to spawn work.
- Con: implies going over an ABI, and cannot be optimized at link-time.
- Con: different implementation may allocate resources for the system scheduler at startup, and then, at the start of main, the implementation is replaced (this is mainly a QOI issue).
- Con: requires strict lifetime and ownership control to be safe, and for the user to do the right thing explicitly.

Compile-time replaceability has the following characteristics:

- Pro: users can do this with a type-def that can be used everywhere and switched.
- Con: potential problems with ODR violations.
- Con: doesn't support shareability across different binaries of the same process

The paper considers compile-time replaceability as not being a valid option because it easily breaks one of the fundamental design principles of a `system_context`, i.e. having one, shared, application-wide execution context, which avoids oversubscription.

We want to obtain feedback from different groups before moving forward with either link-time or run-time replaceability design.

### § 4.3.1. Link-time replaceability is not perfect

The reason we want to also consider the run-time replaceability is because there are some examples (from the practical standpoint) where link-time replaceability might lead to unexpected behavior. All the analysis is done based on the `operator new/operator delete` since it the good example in the C++ standard that has link-time replaceability.

#### § *4.3.1.1. Example on Linux*

Let's consider Linux and its toolchain (GCC, for instance).

Imagine that we have two implementations of the ABI mentioned in § 4.5 ABI for system_scheduler in some `.so`. Let's say one is `libsystem_scheduler_tbb.so` and another one is `libsystem_scheduler_openmp.so`. The problem is whichever library goes first in the command line, it would be a system scheduler to use while the other one silently ignored.

Please compare two different examples below:

```
g++ my_app.cpp -lsystem_scheduler_tbb -lsystem_scheduler_openmp // TBB sche
```

vs

```
g++ my_app.cpp -lsystem_scheduler_openmp -lsystem_scheduler_tbb // OpenMP s
```

which might be unexpected by for the users.

Of course, in the example above with the simple command line everything might look more or less clear but if someone has the complex build system debugging such an issue might be because the possible scenario is that the correctness is not affected, only performance is.

#### § *4.3.1.2. Example on Windows*

On Windows the situation is even worse.

Imagine that you have the application and a DLL that implements the ABI from § 4.5 ABI for system_scheduler. The problem is when you link that very DLL with the application the symbols remain within the DLL and they do not replace the system scheduler default implementation, which is even more unobvious for the users.

#### § *4.3.1.3. Conclusion*

While run-time replaceability has some drawbacks, it lacks the issues described above. When one would change system scheduler with run-time replacement mechanism (if it will be the case), it would work predictably on the different operating systems, similar to `set_terminate` API.

One of the potential mitigations of run-time replaceability issues (such as data races) is that we could say that *"the `system_scheduler` shall be replaced only once and before any outstanding work submitted to it by the application in `main`"* or something similar. It not a formal wording, of course; it's an example to give some ruminations.

## § 4.4. Extensibility

The `std::execution` framework is expected to grow over time. We expect to add time-based scheduling, async I/O, priority-based scheduling, and other for now unforeseen functionality. The `system_context` framework needs to be designed in such a way that it allows for extensibility.

Whatever the replaceability mechanism is, we need to ensure that new features can be added to the system context in a backwards-compatible manner.

There are two levels in which we can extend the system context:

1. Add more types of schedulers, beside the system scheduler.
2. Add more features to the existing scheduler.

The first type of extensibility can easily be solved by adding new getters for the new types of schedulers. Different types of schedulers should be able to be replaced separately; e.g., one should be able to replace the I/O scheduler without replacing the system scheduler. The discussed replaceability mechanisms support this.

The second type of extensibility can also be easily achieved, but, at this point, it's beside of the scope of this paper. Next section provides more details.

## § 4.5. ABI for `system_scheduler`

A proper implementation of the system scheduler that meets all the goals expressed in the paper needs to be divided into two parts: "host" and "backend". The host part implements the API defined in this paper and calls the backend for the actual implementation. The backend provides the actual implementation of the system context (e.g., use Grand Central Dispatch or Windows Thread Pool).

As we need to switch between different backend, we need a "stable" ABI between these two parts.

While standardizing the ABI is going to be challenging, the authors believe that doing that would be beneficial to the users because it creates the portable way of substituting `system_scheduler` for every C++ standard library implementation and for all OS, supporting modern C++ (Linux, Windows, etc.).

Below you can see how the ABI might look like:

```
struct exec_system_scheduler_interface {
  /// The forward progress guarantee of the scheduler.
  ///
  /// 0 == concurrent, 1 == parallel, 2 == weakly_parallel
  uint32_t forward_progress_guarantee;

  /// The size of the operation state object on the implementation side.
  uint32_t schedule_operation_size;
  /// The alignment of the operation state object on the implementation sid
  uint32_t schedule_operation_alignment;

  /// Schedules new work on the system scheduler, calling `cb` with `data`
  /// Returns an object that should be passed to destruct_schedule_operatio
  void* (*schedule)(
    struct exec_system_scheduler_interface* /*self*/,
    void* /*preallocated*/,
    uint32_t /*psize*/,
    exec_system_context_completion_callback_t /*cb*/,
    void* /*data*/);

  /// Destructs the operation state object.
  void (*destruct_schedule_operation)(
```

```
    struct exec_system_scheduler_interface* /*self*/,
    void* /*operation*/);

  /// The size of the operation state object on the implementation side.
  uint32_t bulk_schedule_operation_size;
  /// The alignment of the operation state object on the implementation sid
  uint32_t bulk_schedule_operation_alignment;

  /// Schedules new bulk work of size `size` on the system scheduler, calli
  /// for indices in [0, `size`), and calling `cb` on general completion.
  /// Returns the operation state object that should be passed to destruct_
  void* (*bulk_schedule)(
    struct exec_system_scheduler_interface* /*self*/,
    void* /*preallocated*/,
    uint32_t /*psize*/,
    exec_system_context_completion_callback_t /*cb*/,
    exec_system_context_bulk_item_callback_t /*cb_item*/,
    void* /*data*/,
    long /*size*/);

  /// Destructs the operation state object for a bulk_schedule.
  void (*destruct_bulk_schedule_operation)(
    struct exec_system_scheduler_interface* /*self*/,
    void* /*operation*/);
};
```

This is just an example that is used in the current reference implementation. In principle, it can be C ABI, virtual functions, whatever we agree on.

The implementation of the ABI above can be found in stdexec repository.

The authors would like to obtain feedback from the C++ standard library implementers before advancing a proposal for standardizing the ABI.

## § 4.6. Shareability

One of the motivations of this paper is to stop the proliferation of local thread pools, which can lead to CPU oversubscription. If multiple binaries are used in the same process, we don't want each binary to have its own implementation of system context. Instead, we would want to share the same underlying implementation.

The recommendation of this paper is to leave the details of shareability to be implementation-defined or unspecified.

## § 4.7. Performance

As discussed above, one possible approach to the system context is to implement link-time replaceability. This implies moving across binary boundaries, over some defined API (which should be extensible). A common approach for this is to have COM-like objects. However, the problem with that approach is that it requires memory allocation, which might be a costly operation. This becomes problematic if we aim to encourage programmers to use the system context for spawning work in a concurrent system.

While there are some costs associated with implementing all the goals stated here, we want the implementation of the system context to be as efficient as possible. For example, a good implementation should avoid memory allocation for the common case in which the default implementation is utilized for a platform.

This paper cannot recommend the specific implementation techniques that should be used to maximize performance; these are considered Quality of Implementation (QOI) details.

## § 4.8. Lifetime

Underneath the `system_context`, there is a singleton of some sort. We need to specify the lifetime of this object and everything that derives from it.

The paper mandates that the lifetime of any `system_context` to fully be contained the lifetime of `main()`.

During the design of this paper the authors considered a more relaxed model: allow the `system_context` to be created before `main()` (as part of static initialization), but still mandate that all access to cease before the end of `main()`; this model is somehow similar to the Intel Threading building blocks (oneTBB) model. While the relaxed model can prove sometime useful, we considered it to be a dangerous path, for the following reasons:

- The scope of `system_context` should be deterministic with respect to global static objects and main; this allows people to reason about the application.
- We want to guarantee the access to static objects to all the work spawned on `system_context`.

- A global static constructor may add work on the `system_context`, thus, it may block on the completion of that work anytime until destruction; this would imply possibly blocking the termination of the program, after `main()` is complete.
- For replaceability: we want to guarantee the access to static objects for any replacement of `system_context`.
- There may be circular dependencies between user-supplied `system_context` and the system allocator.

If we start with this stricter path, we can always relax it later. On other hand, if we start with a more relaxed model, it will be harder to make it stricter after being used in production.

## § 4.9. Need for the `system_context` class

Our goal is to expose a global shared context to avoid oversubscription of threads in the system and to efficiently share a system thread pool. Underneath the `system_context` there is a singleton of some sort, potentially owned by the OS.

The question is how we expose the singleton. We have a few obvious options:

- Explicit context objects, as we've described in R2 and R3 of this paper, where a `system_context` is constructed as any other context might be, and refers to a singleton underneath.
- A global `get_system_context()` function that obtains a `system_context` object, or a reference to one, representing the singleton explicitly.
- A global `get_system_scheduler()` function that obtains a scheduler from some singleton system context, but does not explicitly expose the context.

The `get_system_context()` function returning by value adds little, it's equivalent to direct construction. `get_system_context()` returning by reference and `get_system_scheduler()` have a different lifetime semantic from directly constructed `system_context`.

The main reason for having an explicit by-value context is that we can reason about lifetimes. If we only have schedulers, from `get_system_context().get_scheduler()` or from `get_system_scheduler()` then we have to think about how they affect the context lifetime. We might want to reference count the context, to ensure it outlives the schedulers, but this adds cost to each scheduler use, and to any downstream sender produced from the scheduler as well that is logically dependent on the scheduler. We could alternatively not reference count and assume the context out-

lives everything in the system, but that leads quickly to shutdown order questions and potential surprises.

By making the context explicit we require users to drain their work before they drain the context. In debug builds, at least, we can also add reference counting so that destruction of the context before work completes reports a clear error to ensure that people clean up. That is harder to do if the context is destroyed at some point after main completes. This lifetime question also applies to construction: we can lazily construct a thread pool before we first use a scheduler to it.

For this reason, and consistency with other discussions about structured concurrency, we opt for an explicit context object here.

## § 4.10. Priorities

It's broadly accepted that we need some form of priorities to tweak the behaviour of the system context. This paper does not include priorities, though early drafts of R2 did. We had different designs in flight for how to achieve priorities and decided they could be added later in either approach.

The first approach is to expand one or more of the APIs. The obvious way to do this would be to add a priority-taking version of `system_context::get_scheduler()`:

```
implementation-defined-system_scheduler get_scheduler();
implementation-defined-system_scheduler get_scheduler(priority_t priority);
```

This approach would offer priorities at scheduler granularity and apply to large sections of a program at once.

The other approach, which matches the receiver query approach taken elsewhere in [P2300R9] is to add a `get_priority()` query on the receiver, which, if available, passes a priority to the scheduler in the same way that we pass an `allocator` or a `stop_token`. This would work at task granularity, for each `schedule()` call that we connect a receiver to we might pass a different priority.

In either case we can add the priority in a separate paper. It is thus not urgent that we answer this question, but we include the discussion point to explain why they were removed from the paper.

## § 4.11. Reference implementation

The authors prepared a reference implementation in stdexec

A few key points of the implementation:

- The implementation is divided into two parts: "host" and "backend". The host part implements the API defined in this paper and calls the backend for the actual implementation. The backend provides the actual implementation of the system context.
- Allows link-time replaceability for `system_scheduler`. Provides examples on doing this.
- Defines a simple C API between the host and backend parts. This way, one can easily extend this interface when new features need to be added to `system_context`.
- Uses preallocated storage on the host side, so that the default implementation doesn't need to allocate memory on the heap when adding new work to `system_scheduler`.
- Guarantees a lifetime of at least the duration of `main()`.
- As the default implementation is created outside of the host part, it can be shared between multiple binaries in the same process.
- TODO: Use OS scheduler for implementation.

## § 4.12. Addressing received feedback

### § 4.12.1. Allow for system context to borrow threads

Early feedback on the paper from Sean Parent suggested a need for the system context to support a configuration where it carries no threads of its own and takes over the main thread. While in [P2079R2] we proposed `execute_chunk` and `execute_all`, these enforce a particular implementation on the underlying execution context. Instead, we simplify the proposal by removing this functionality and assuming that it is implemented by link-time or run-time replacement of the context. We assume that the underlying mechanism to drive the context, should one be necessary, is implementation-defined. This allows for custom hooks into an OS thread pool, or a simple `drive()` method in main.

As we discussed previously, a separate paper is supposed to take care of the drive-ability aspect.

### § 4.12.2. Allow implementations to use Grand Central Dispatch and Windows Thread Pool

In the current form of the paper, we allow implementations to define the best choice for implementing the system context for a particular system. This includes using Grand Central Dispatch on Apple plat-

forms and Windows Thread Pool on Windows.

In addition, we propose implementations to allow the replaceability of the system context implementation. This means that users should be allowed to write their own system context implementations that depend on OS facilities or a necessity to use some vendor (like Intel) specific solutions for parallelism.

#### 4.12.3. Priorities and elastic pools

Feedback from Sean Parent:

> There is so much in that proposal that is not specified. What requirements are placed on the system scheduler? Most system schedulers support priorities and are elastic (i.e., blocking in the system thread pool will spin up additional threads to some limit).

The lack of details in the specification is intentional, allowing implementers to make the best compromises for each platform. As different platforms have different needs, constraints, and optimization goals, the authors believe that it is in the best interest of the users to leave some of these details as Quality of Implementation (QOI) details.

#### 4.12.4. Implementation-defined may make things less portable

Some feedback gathered during discussions on this paper suggested that having many aspects of the paper to be implementation-defined would reduce the portability of the system context.

While it is true that people that would want to replace the system scheduler will have a harder time doing so, this will not affect the users of the system scheduler. They would still be able to the use system context and system scheduler without knowing the implementation details of those.

We have a precedence in the C++ standard for this approach with the global allocator.

## 5. Questions to ask LEWG/SG1/vendors

### 5.1. What type of replaceability we want?

Do we want link-time replaceability or run-time replaceability?

### § 5.2. Do we want to standardize an ABI for `system_scheduler` (as opposed to leaving this to be implementation defined)?

Proposed answer: YES.

To allow users to easily change on the scheduler, they need to know how to replace the system scheduler backend. The best way to do that is if we standardize the interface that the system scheduler has.

Tradeoffs:

- Pro: allow users to have a portable replacement mechanism of `system_scheduler` across all standard library implementations.
- Pro: allow users to react faster to new technologies (without needing to wait for a new C++ release cycle).
    - I.e., similar to adopting `io_uring`, which got heavy adoption in a short amount of time
- Pro: increased portability
- Con: slightly larger interface needed to be standardized, and this is not something common for the C++ standard.
- Con: Might leave C++ standard library implementors less freedom on best interfaces for the targeted platforms. It should be possible to mitigate this Con by working with C++ standard library vendors and understanding their needs.

### § 5.3. Do we want to allow system scheduler to be used before start of `main()` ?

Proposed answer: NO. We believe that this would create more problems that it actually solves.

## § 6. Examples

As a simple parallel scheduler we can use it locally, and `sync_wait` on the work to make sure that it is complete. With forward progress delegation this would also allow the scheduler to delegate work to the blocked thread. This example is derived from the Hello World example in [P2300R9]. Note that it only adds a well-defined context object, and queries that for the scheduler. Everything else is unchanged about the example.

```
using namespace std::execution;

system_context ctx;
```

```
scheduler auto sch = ctx.scheduler();

sender auto begin = schedule(sch);
sender auto hi = then(begin, []{
    std::cout << "Hello world! Have an int.";
    return 13;
});
sender auto add_42 = then(hi, [](int arg) { return arg + 42; });

auto [i] = this_thread::sync_wait(add_42).value();
```

We can structure the same thing using `execution::on`, which better matches structured concurrency:

```
using namespace std::execution;

system_context ctx;
scheduler auto sch = ctx.scheduler();

sender auto hi = then(just(), []{
    std::cout << "Hello world! Have an int.";
    return 13;
});
sender auto add_42 = then(hi, [](int arg) { return arg + 42; });

auto [i] = this_thread::sync_wait(on(sch, add_42)).value();
```

The `system_scheduler` customises `bulk`, so we can use `bulk` dependent on the scheduler. Here we use it in structured form using the parameterless `get_scheduler` that retrieves the scheduler from the receiver, combined with `on`:

```
auto bar() {
  return
    ex::let_value(
      ex::get_scheduler(),          // Fetch scheduler from receiver.
      [](auto current_sched) {
        return bulk(
          current_sched.schedule(),
          1,                        // Only 1 bulk task as a lazy way of ma
          [](auto idx){
            std::cout << "Index: " << idx << "\n";
```

```
          })
      });
}

void foo()
{
  using namespace std::execution;

  system_context ctx;

  auto [i] = this_thread::sync_wait(
    on(
      ctx.scheduler(),                // Start bar on the system_scheduler
      bar()))                         // and propagate it through the rece
    .value();
}
```

Use `async_scope` and a custom system context implementation linked in to the process (through a mechanism undefined in the example). This might be how a given platform exposes a custom context. In this case we assume it has no threads of its own and has to take over the main thread through an custom `drive()` operation that can be looped until a callback requests `exit` on the context.

```
using namespace std::execution;

system_context ctx;

int result = 0;

{
  async_scope scope;
  scheduler auto sch = ctx.scheduler();

  sender auto work =
    then(just(), [&](auto sched) {

      int val = 13;

      auto print_sender = then(just(), [val]{
        std::cout << "Hello world! Have an int with value: " << val << "\n"
      });
```

```
      // spawn the print sender on sched to make sure it
      // completes before shutdown
      scope.spawn(on(sch, std::move(print_sender)));

      return val;
    });

  scope.spawn(on(sch, std::move(work)));

  // This is custom code for a single-threaded context that we have replace
  // We need to drive it in main.
  // It is not directly sender-aware, like any pre-existing work loop, but
  // does provide an exit operation. We may call this from a callback chain
  // after the scope becomes empty.
  // We use a temporary terminal_scope here to separate the shut down
  // operation and block for it at the end of main, knowing it will complet
  async_scope terminal_scope;
  terminal_scope.spawn(
    scope.on_empty() | then([](my_os::exit(ctx))));
  my_os::drive(ctx);
  this_thread::sync_wait(terminal_scope);
};

// The scope ensured that all work is safely joined, so result contains 13
std::cout << "Result: " << result << "\n";
```

# § References

## § Informative References


**[P0443R14]**

Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysen, H. Carter Edwards, Gordon Brown, D. S. Hollman. *A Unified Executors Proposal for C++*. 15 September 2020. URL: https://wg21.link/p0443r14

**[P2079R1]**

Ruslan Arutyunyan, Michael Voss. *Parallel Executor*. 15 August 2020. URL: https://wg21.link/p2079r1

**[P2079R2]**

Lee Howes, Ruslan Arutyunyan, Michael Voss. *System execution context*. 15 January 2022. URL: https://wg21.link/p2079r2

**[P2300R9]**

Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. *`std::execution`*. 2 April 2024. URL: https://wg21.link/p2300r9

**[P3109R0]**

Lewis Baker, Eric Niebler, Kirk Shoop, Lucian Radu. *A plan for std::execution for C++26*. 12 February 2024. URL: https://wg21.link/p3109r0

**[P3149R2]**

Ian Petersen, Ján Ondrušek; Jessica Wong; Kirk Shoop; Lee Howes; Lucian Radu Teodorescu;. *async_scope — Creating scopes for non-sequential concurrency*. 20 March 2024. URL: https://wg21.link/p3149r2